Architecture

1. Architectural style

The main financial application is kept as a **modular monolith**. The goal is to avoid turning Wallet, Transaction and Ledger components that participate in one atomic money movement into a distributed-transaction problem through premature microservice decomposition.

Main solution projects:

```
src/
  FinWallet.Api
  FinWallet.Application
  FinWallet.Domain
  FinWallet.Infrastructure
  FinWallet.Shared.Contracts
  FinWallet.Shared.Web
  FinWallet.Gateway
simulators/
  FakeBank.Api
  FakeFraud.Api
  FakeCutoff.Api
  FakeCampaign.Api
  FakeCommunication.Api
tests/
  FinWallet.Application.Tests
```

2. Layer responsibilities

Domain: entities, value objects, invariants, state transitions and financial rules. It has no knowledge of HTTP, SQL or provider DTOs.

Application: use-case orchestration and ports/interfaces. It uses Domain but does not depend on Infrastructure implementations.

Infrastructure: MSSQL, Redis, JWT token issuance, provider HttpClient adapters and persistence implementations.

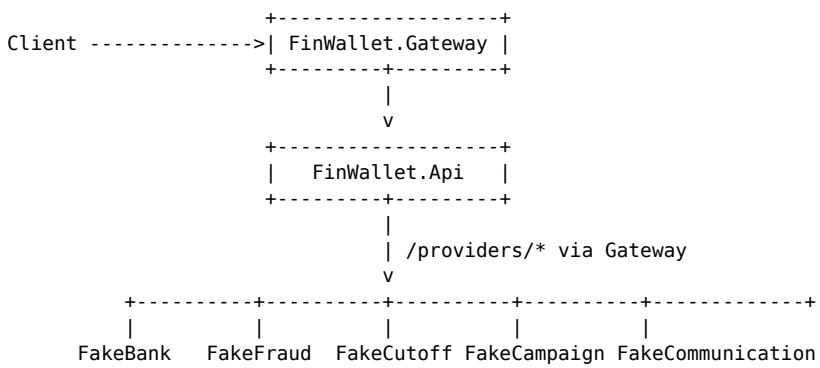
Api: controllers, HTTP-contract mapping, authentication/authorization composition and exception mapping.

Shared.Contracts: common HTTP envelope and stable transport contracts shared with simulators.

Shared.Web: shared hosting concerns such as Swagger, rate limiting, Kestrel limits, CORS, security headers and internal-service-key checks.

Gateway: YARP routes/clusters, edge JWT validation, internal-service authorization, load balancing and traffic controls.

3. Runtime topology



Clients normally do not call backend services directly. FinWallet.Api also does not call provider simulators directly; it uses Gateway provider routes.

4. Trust boundaries

1. **Client -> Gateway:** public authentication boundary. Protected /api/* routes require JWT.
2. **Gateway -> FinWallet.Api:** Gateway adds a separate downstream service credential. The API independently validates JWT and ownership.
3. **FinWallet.Api -> Gateway provider route:** requires InternalServiceKey.
4. **Gateway -> simulator:** requires DownstreamServiceKey.

This prevents a bypassed Gateway from automatically turning a backend business endpoint into a trusted endpoint.

5. Financial transaction boundary

A wallet-to-wallet transfer commits the following in one MSSQL transaction:

- durable idempotency;
- source-wallet debit;
- destination-wallet credit;
- FinancialTransaction;
- LedgerJournal;
- LedgerEntries.

External HTTP is never executed while that SQL transaction is open.

6. Dependency direction

```
Api -> Application -> Domain
Api -> Infrastructure -> Application/Domain
Gateway -> Shared.Web/Shared.Contracts
Simulators -> Shared.Web/Shared.Contracts
```

Domain must not depend on Infrastructure/API. Application must not know provider-specific DTOs or concrete SQL implementations.

7. Why not microservices?

Today's boundaries optimize for correctness and comprehensibility rather than independent deployment. Service decomposition can be reconsidered when domain boundaries are stable and there is a real need for independent scaling or team ownership. Splitting components such as Wallet and Ledger that require the same atomic commit is particularly expensive.